

Introduction: What Is Computational Thinking?

1. Meaning of Computational Thinking

Computational thinking is **a way of solving problems by breaking them into clear, repeatable steps**. It helps us organize our thoughts so that another person or a computer can understand and follow them.

Computation thinking is **not the same as programming**. Programming means writing instructions in a particular language such as Python, Java, or JavaScript.

First we perform Computational thinking later we decide what the steps should be, and then we may write those steps as a program.

A simple example

Suppose you want to **find a book** in a **bookshelf**.

You might:

1. Decide which shelf to check first.
2. Look at the books one by one.
3. Compare each book with the title you want.
4. Stop when you find the book.
5. Report that the book was not found if you reach the end.

This is already a small computational process. You are following a method rather than searching randomly.

Everyday activities that use computational thinking

You use similar thinking when you:

- Search for an item in a supermarket.
- Check tasks on a to-do list.
- Tick off friends from an invitation list.
- Arrange clothes on several clotheslines.
- Pack objects into suitcases.
- Find a route from one city to another.
- Search your phone for a contact.
- Compare prices before buying a product.

The objects change, but the underlying method may remain the same.

Reusable patterns

A **pattern** is a method that **can be reused in different situations**.

For example, consider arranging objects on shelves:

1. Choose the largest object.
2. Place it at one end.
3. Choose the next largest object.
4. Place it beside the previous object.
5. Continue until the shelf is full.
6. Move to the next shelf.

The same pattern could be used for:

- Clothes on clotheslines.
- Books on bookshelves.
- Travel items in suitcases.
- Files in storage folders.

Only the names of the objects and locations change. The basic process remains similar.

Visual description

Imagine a diagram with three columns:

- **Objects:** clothes, books, or travel items.
- **Destination:** clothesline, shelf, or suitcase.
- **Common method:** arrange from largest to smallest and use space carefully.

This diagram would show that the same solution pattern can work in different situations.

Common mistakes

- Thinking computational thinking only applies to computers.
- Believing that every problem must immediately be converted into code.
- Giving instructions that are too vague, such as “arrange the room properly.”
- Giving unnecessary details when the person already understands the task.
- Solving the same type of problem from the beginning every time instead of reusing a known method.

Practice thinking

Think about preparing for a college examination. What common pattern could you reuse for each subject?

Possible steps might include:

1. List the topics.
2. Mark the topics you already understand.
3. Study the difficult topics first.
4. Practice questions.
5. Review mistakes.
6. Repeat until the examination.

Recap: Computational thinking is a structured way to solve problems. Its main strength is recognizing reusable patterns and expressing them as clear steps that people or computers can follow.

2. Stepwise Problem Solving

The basic unit of computational thinking is a **step**. A step is one action performed during a problem-solving process.

For example, when searching a supermarket shelf, one step might be: Examine one item and check its label.

You then repeat this step for other items.

What makes a good step?

The level of detail depends on the person or system receiving the instructions.

If someone already knows how to arrange furniture, you might say:

Arrange the furniture for the meeting.

If the person has no experience, you must explain the task in more detail:

1. Find out how many people will attend.
2. Determine how many chairs are needed.
3. Determine how many tables are needed.
4. Place the tables around the room.
5. Put the required number of chairs around each table.
6. Leave enough space for people to walk.

A computer usually needs very precise instructions because it cannot rely on common sense in the same way a human can.

Step-by-step method for solving a problem

When facing a new problem, use this approach:

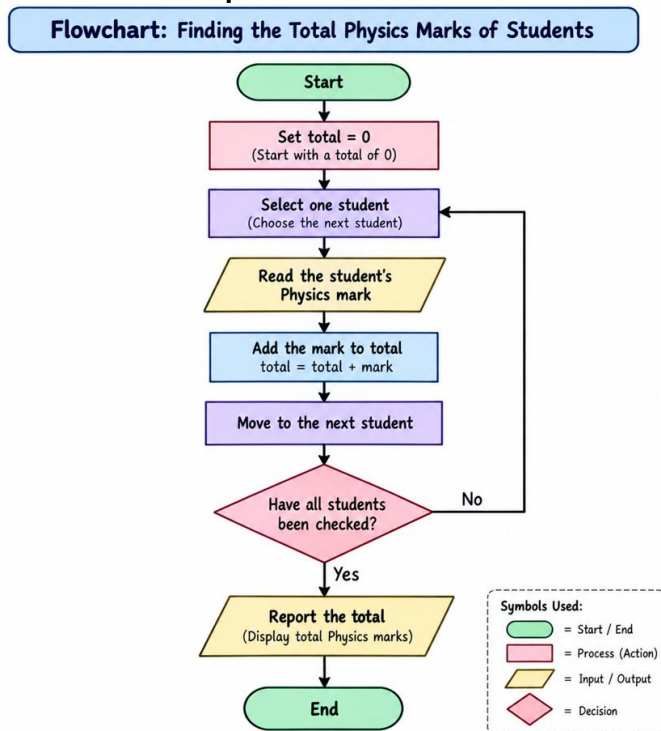
1. **Understand the goal.**
Clearly state what answer or result is needed.
2. **Identify the objects involved.**
These may be students, bills, words, books, or numbers.
3. **Break the work into small actions.**
Decide what must be done first, next, and last.
4. **Look for repetition.**
Ask whether the same action must be performed many times.
5. **Identify decisions.**
Ask whether some objects should be included and others ignored.
6. **Track important information.**
Decide what values must be remembered while working.
7. **Decide when to stop.**
A process should not continue forever.

Example: Finding the total marks of students

Suppose a class has several students, and you want the total Physics marks.

1. Start with a total of 0.
2. Select one student.
3. Read the student's Physics mark.
4. Add it to the total.
5. Move to the next student.
6. Repeat until every student has been checked.
7. Report the total.

Visual description



Common mistakes

- Missing one item.
- Checking the same item more than once.
- Using the wrong field, such as adding Chemistry marks instead of Physics marks.
- Forgetting to initialize the total to 0.
- Forgetting to define the stopping condition.
- Assuming that a vague instruction will always be understood correctly.

Practice thinking

Write the steps for finding your name in a list of students. Should the search stop when your name is found, or should it continue to the end? Explain why.

Recap: A problem becomes easier when it is expressed as small, ordered actions. Good instructions identify the goal, describe each action clearly, handle repetition and decisions, and specify when the process ends.

3. Logical Thinking and Computational Thinking

Computational thinking uses logic, but the two ideas are not identical.

Logical thinking

Logical thinking often uses known rules to reach a specific conclusion. It expects statements to be clearly true or false.

For example:

- $2+2=4$ is true.
- $2-3=0$ is false.

Logical thinking may also prove that an idea is impossible by showing that it leads to a contradiction.

Computational thinking

Computational thinking focuses on constructing a procedure that produces a useful result.

It is usually more practical than perfectly exact. A solution may be acceptable if it completes the task reasonably well.

Example: Choosing the best student

There may not be one student who is best in every subject. One student may be strongest in Mathematics, another in Physics, and another in Chemistry.

A purely logical discussion might say:

There is no single best student because different students have different strengths.

A computational approach might define a rule:

1. Add the marks in all subjects.
2. Compare the total marks.
3. Select the student with the highest total.

This rule may not capture every idea of “best,” but it produces a clear and usable answer.

Logical Thinking and Computational Thinking

Computational thinking uses logic, but the two ideas are not identical.

Logical Thinking

- Uses known rules to reach a specific conclusion.
- Expects statements to be clearly true or false.
- May also prove that an idea is impossible by showing that it leads to a contradiction.

For example:

- $2 + 2 = 4$ is true. ✓
- $2 - 3 = 0$ is false. ✗

Is this true or false?

Logical thinking can also prove something is impossible by showing a contradiction.

Computational Thinking

- Focuses on constructing a procedure that produces a useful result.
- Usually more practical than perfectly exact.
- A solution may be acceptable if it completes the task reasonably well.

How can I solve this step by step?

It's about creating a method that works!

Example: Choosing the Best Student

There may not be one student who is best in every subject. One student may be strongest in Mathematics, another in Physics, and another in Chemistry.

Student	Mathematics	Physics	Chemistry
Anita	95	80	70
Rahul	75	92	78
Meera	68	74	96

Logical Thinking Approach

Who is the best student?

A purely logical discussion might say:

"There is no single best student because different students have different strengths."

Computational Thinking Approach

Define a clear rule and follow a procedure:

1. Add the marks in all subjects.
2. Compare the total marks.
3. Select the student with the highest total.

Student	Mathematics	Physics	Chemistry	Total
Anita	95	80	70	245
Rahul	75	92	78	245
Meera	68	74	96	238

This rule may not capture every idea of "best," but it produces a clear and usable answer.

Deductive and inductive thinking

Deductive vs Inductive Thinking (EL10)

Two ways of thinking from facts to rules, or from rules to facts!

Deductive Thinking

Goes from **GENERAL** (laws) to **SPECIFIC** (application).

We start with a general rule or law, and apply it to a particular case to get a specific conclusion.

General rule (law)

Apply to a specific case

Specific conclusion

Example (Maths)

General rule (theorem): Squares of integers are positive.

Apply to a specific number (e.g., -2): $(-2)^2 = 4$

Conclusion: 4 is positive. So the theorem is true for -2 .

We start with a known rule and use it!

Inductive Thinking

Goes from **SPECIFIC** (observations) to **GENERAL** (rule).

We start with specific examples or observations, and look for a pattern to form a general rule.

Specific observations

Look for a pattern

General rule (conclusion)

Example (Cows)

This cow produces milk.
This cow produces milk.
This cow produces milk.

General rule (inductive conclusion): All cows produce milk.

We see examples and make a general rule!

Inductive Thinking in Computational Thinking (Repetition)

In computational thinking, we often do something once, find a general step (pattern), and then repeat it many times.

Example: Arranging tables and chairs for a party

1. Do it for one table (specific observation)



Arrange one table with chairs.

2. General step (pattern)

Arrange any table with chairs.

3. Repeat for all tables



Use the same step for every table.

Example: Hanging clothes on a clothesline

1. Do it for one item (specific observation)



Hang one cloth item.

2. General step (pattern)

Pick the largest item and hang it next to the previous one.

3. Repeat for all items



Use the same step for every item.

★ Key Takeaways

- Deductive thinking: general $\rightarrow$ specific (use a rule to solve a particular case).
- Inductive thinking: specific $\rightarrow$ general (find a rule from examples).
- Repetition in computational thinking is inductive: we do it once, make a general step, and repeat it many times.
- These ideas help us solve real-life problems efficiently.

Observe
 $\rightarrow$ Find a Pattern
 $\rightarrow$ Repeat
 $\rightarrow$ Solve Bigger Problems!

Deductive thinking moves from a general rule to a specific case.

Example:

1. All integers have a square that is non-negative.
2. -2 is an integer.
3. Therefore, $(-2)^2$ is non-negative.

Inductive thinking moves from specific observations to a general pattern.

Example:

1. One cow produces milk.
2. Another cow produces milk.
3. Several other cows produce milk.
4. We form a general belief about cows.

Computational thinking often uses induction because we observe one example, identify a method, and reuse it for many similar objects.

Common mistakes

- Treating computational thinking as only mathematical proof.
- Assuming that a solution must be perfect in every possible sense.
- Confusing “a useful procedure” with “a proof.”
- Believing that a computer can automatically understand an incomplete rule.

Practice thinking

Suppose you want to choose the “best” mobile phone. What measurable rule could you use?

Possible rules include:

- Lowest price.
- Longest battery life.
- Best camera.
- Highest combined score across several features.

Which rule would be most useful for your purpose?

Recap: Logical thinking helps us reason correctly, while computational thinking helps us build procedures that produce results. Computational thinking is practical and focuses on clear, repeatable construction rather than only proving statements.

4. Datasets and Data Elements

A **dataset** is a collection of related information.

Examples include:

- A list of student marks.
- A collection of shopping bills.
- A paragraph divided into words.
- A train timetable.
- A list of rectangles and their measurements.

A single item in a dataset is called a **data element**.

For example, in a student dataset, one student’s marks card is one data element.

Example: Student data element

A student record may contain:

- Identification number.
- Name.
- Gender.
- Date of birth.
- Town or city.
- Mathematics marks.

- Physics marks.
- Chemistry marks.
- Total marks.

A dataset may contain many such records.

Datasets and Data Elements

A dataset is a collection of related information. A single item in a dataset is called a data element.

Many items together make a dataset!
Each item is a data element.

1. What is a Dataset?

A dataset is a collection of related information.

Examples:

- A list of student marks
- A collection of shopping bills
- A paragraph divided into words
- A train timetable
- A list of rectangles and their measurements

The items may be numbers, words, dates, or other information, but they are related.

2. What is a Data Element?

A single item in a dataset is called a **data element**.

Example: Student data element

A student record may contain:

- Identification number
- Name
- Gender
- Date of birth
- Town or city
- Mathematics marks
- Physics marks
- Chemistry marks
- Total marks

One student record
(one data element)

ID : 1
Name : Asha
Gender : Female
DOB : 12-05-2006
City : Chennai
Maths : 78
Physics : 82
Chemistry : 75
Total : 235

A dataset may contain many such records (for example, one record for each student).

3. Questions We Can Ask About a Dataset

Datasets help us answer different kinds of questions.

For a student dataset:

- Who scored the highest in Mathematics?
- What is the class average?
- Are there students born on the same day?
- Are the girls' average marks higher than the boys' average?
- Which students performed well in all subjects?

For a shopping dataset:

- Which customer spent the most?
- Which shop earned the highest revenue?
- Which product category sold the most?
- Which customers repeatedly visited the same shop?

For a words dataset:

- How many words are in each sentence?
- Which words occur most often?
- Which words are verbs?
- How many letters does each word contain?

4. Physical-Card Idea

Imagine each data element written on a separate card.

Student cards

Shopping bill cards

Word cards

Step-by-step process:

- 1 Keep unexamined cards in one pile.
- 2 Pick one card.
- 3 Examine it.
- 4 Record useful information.
- 5 Move it to a "seen" pile.

Unseen pile

Examine card

Seen pile

This makes the process easy to visualize before writing code.

5. Table Representation

The same cards can be represented as rows in a table.

ID	Name	City	Maths	Physics	Chemistry	Total
1	Asha	Chennai	78	82	75	235
2	Ravi	Salem	65	70	80	215

Each row is one data element (one student record).

Each column describes one property of that element (e.g., name, city, marks).

You can have many more rows (records) in the dataset.

Key Takeaways

- A dataset is a collection of related information.
- A data element is a single item in the dataset.
- We can ask many useful questions about a dataset.
- The physical-card idea helps us understand the process.
- The same data can be shown as a table, with rows as data elements and columns as properties.

Organize Data.
Ask Questions.
Find Useful Answers!

Questions we can ask about a dataset

For a student dataset:

- Who scored the highest in Mathematics?
- What is the class average?
- Are there students born on the same day?
- Are the girls' average marks higher than the boys' average?
- Which students performed well in all subjects?

For a shopping dataset:

- Which customer spent the most?
- Which shop earned the highest revenue?
- Which product category sold the most?
- Which customers repeatedly visited the same shop?

For a words dataset:

- How many words are in each sentence?
- Which words occur most often?
- Which words are verbs?

- How many letters does each word contain?

Physical-card idea

One useful way to learn computational thinking is to imagine each data element written on a separate card.

For example:

- One student record per card.
- One shopping bill per card.
- One word per card.

You can then:

1. Keep unexamined cards in one pile.
2. Pick one card.
3. Examine it.
4. Record useful information.
5. Move it to a “seen” pile.

This makes the process easy to visualize before writing code.

Table representation

The same cards can be represented as rows in a table.

ID	Name	City	Maths	Physics	Chemistry	Total
1	Asha	Chennai	78	82	75	235
2	Ravi	Salem	65	70	80	215

Each row is one data element. Each column describes one property of that element.

Visual description

Sketch a table with:

- One row for each student.
- One column for each field.
- An arrow moving down the rows.
- A separate box labelled **Answer** where the result is stored.

This helps you see how a procedure visits the dataset one element at a time.

Common mistakes

- Confusing a dataset with a single data element.
- Mixing values from different rows.
- Forgetting what each column represents.
- Assuming that all datasets are already clean and correct.
- Changing original data unnecessarily during processing.

Practice thinking

Create a small dataset of five mobile applications. What fields could each record contain?

Possible fields:

- App name.
- Category.
- Size.
- Number of downloads.
- Rating.
- Developer name.

Recap: A dataset is a collection of related data elements. Once data is organized into cards or table rows, we can systematically examine it and ask useful questions.
